

CFG, Parse tree, and ambiguous grammar

Jianguo Lu

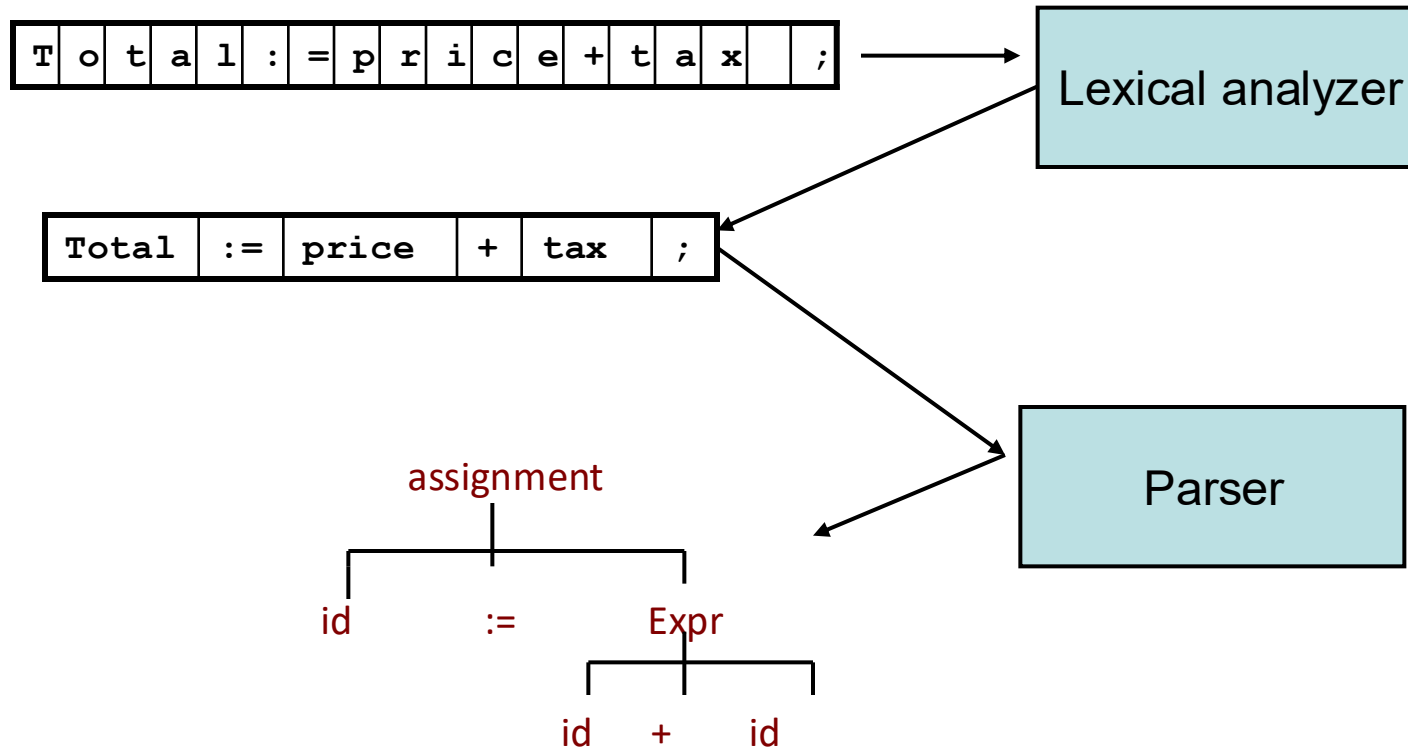
School of Computer Science

University of Windsor

The limit of regular expression (regular grammar)

- Can you use RE to capture the following strings?
 - Nested quotes
 - "Jenny said " yes" when I asked her to marry me."
 - Palindromes
 - step on no pets
 - 202202
 - 20200202

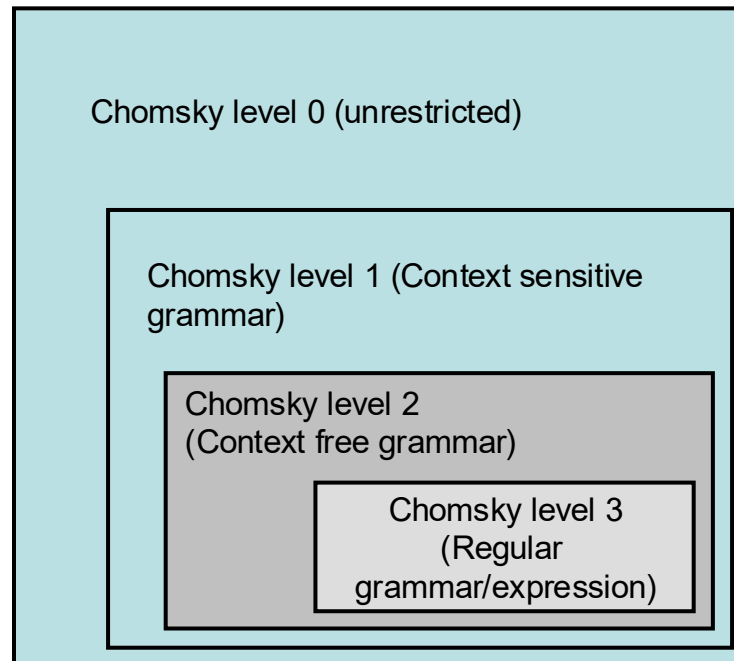
Where we are



Chomsky hierarchy

- A grammar is a 4-tuple $G = (\Sigma, N, P, S)$
 - Σ is a finite set of terminal symbols;
 - N is a finite set of nonterminal symbols;
 - P is a set of productions;
 - S (from N) is the start symbol.
- P can be
$$\alpha_1 \alpha_2 \alpha_3 \dots \alpha_n \rightarrow \beta_1 \beta_2 \beta_3 \dots \beta_m$$
- **Level 3: Regular grammar**
 - $\alpha \rightarrow \beta$ or $\alpha \rightarrow \beta_1 \beta_2$
- **Level 2: Context free grammar**
 - $\alpha \rightarrow \beta_1 \beta_2 \beta_3 \dots \beta_m$
- **Level 1: Context sensitive grammar**
 - $n < m$. The number of symbols on the lhs must not exceed the number of symbols on the rhs
- **Level 0: unrestricted grammar**

Chomsky hierarchy



- The more powerful the grammar, the more complex the program required to recognize the legal inputs.

Grammar examples

- Regular grammar

- $L = \{w \mid w \text{ consists of arbitrary number of 'a's and 'b's}\}$
- Grammar: $S \rightarrow a \mid b \mid aS \mid bS$
- Example sentence: “abb” is a legal sentence in this language
- Derivation:

$S \rightarrow aS$
 $\rightarrow abS$
 $\rightarrow abb$

Context free grammar example

- Context free grammar example
 - Language $L = \{a^n b^n \mid n > 0\}$
 - Grammar: $S \rightarrow ab \mid aSb$
 - Notice the difference with regular grammar.
 - Example sentence: “aaabbb” is a legal sentence in this language
 - Derivation:
 - $S \rightarrow a S b$
 - $\rightarrow a a S b b$
 - $\rightarrow a a a b b b$

Characterize different types of grammars

- Regular grammar
 - Being able to count one item
- Context free grammar
 - being able to count pairs of items
 - $a^n b^n$
- Context sensitive grammar
 - Being able to count arbitrarily;
 - $a^n b^n c^n$

Implications of different grammars in applications

- Regular grammar
 - Recognize words
- Context free grammar
 - Recognize pairs of parenthesis
 $((a+b) * c)/2$
 - Recognize blocks

```
{ statement1;
  { statement2;
    statement3;
  }
}
```
- Context sensitive grammar

Grammar for balanced parentheses

- Valid balanced parentheses:

- $()$
- $((())), (((())) \dots$
- $()((())) \dots$

- The grammar

$S \rightarrow ()$

$S \rightarrow (S)$

$S \rightarrow SS$

Context free grammars and languages

- Many languages are not regular. Thus we need to consider larger classes of languages;
- Context Free Language (CFL) played a central role in natural languages since 1950's (Chomsky) and in compilers since 1960's (Backus);
- Context Free Grammar (CFG) is the basis of BNF syntax;
- CFG is increasingly important for XML and DTD (XML Schema).

Informal Example of CFG

- Palindrome:
 - Madam, I'm adam.
 - A man, a plan, a canal, panama!
- Consider $L_{\text{pal}} = \{w \mid w \text{ is a palindrome on symbols } 0 \text{ and } 1\}$
- Example: 1001, 11 are palindromes
- How to represent palindrome of any length?
- Basis: ϵ , 0 and 1 are palindromes;
 1. $P \rightarrow \epsilon$
 2. $P \rightarrow 0$
 3. $P \rightarrow 1$
- Induction: if w is palindrome, so are $0w0$ and $1w1$. nothing else is a palindrome.
 1. $P \rightarrow 0P0$
 2. $P \rightarrow 1P1$

The informal example (cont.)

- CFG is a formal mechanism for definitions such as the one for L_{pal}
 1. $P \rightarrow \epsilon$
 2. $P \rightarrow 0$
 3. $P \rightarrow 1$
 4. $P \rightarrow 0P0$
 5. $P \rightarrow 1P1$
- 0 and 1 are terminals
- P is a variable (or nonterminal, or syntactic category)
- P is also the start symbol.
- 1-5 are productions (or rules)

Formal definition of CFG

- A CFG is a 4-tuple $G=(\Sigma,N,P,S)$ where
 1. Σ is a finite set of terminals;
 2. N is finite set of variables (non-terminals);
 3. P is a finite set of productions of the form $A \rightarrow \alpha$, where A is a nonterminal and α consists of symbols from Σ and N ;
 1. A is called the head of the production;
 2. α is called the body of the production;
 4. S is a designated non-terminal called the start symbol.
- **Example:**
 - $G_{pal}=(\{0,1\}, \{P\}, A,P)$, where $A =\{P \rightarrow \varepsilon, P \rightarrow 0, P \rightarrow 1, P \rightarrow 0P0, P \rightarrow 1P1\}$
- **Sometimes we group productions with the same head.**
 - e.g., $A=\{P \rightarrow \varepsilon \mid 0 \mid 1 \mid 0P0 \mid 1P1\}$.

Another CFG example: arithmetic expression

- $G = (\{+, *, (,), a, b, 0, 1\}, \{E, R, S\}, P, E)$

1. $E \rightarrow R$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $R \rightarrow aS$
6. $R \rightarrow bS$
7. $S \rightarrow \varepsilon$
8. $S \rightarrow aS$
9. $S \rightarrow bS$
10. $S \rightarrow 0S$
11. $S \rightarrow 1S$

Two level descriptions

- Context-free syntax of arithmetic expressions

1. $E \rightarrow R$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$

- Lexical syntax of arithmetic expressions

1. $R \rightarrow aS$
2. $R \rightarrow bS$
3. $S \rightarrow \epsilon$
4. $S \rightarrow aS$
5. $S \rightarrow bS$
6. $S \rightarrow 0S$
7. $S \rightarrow 1S$

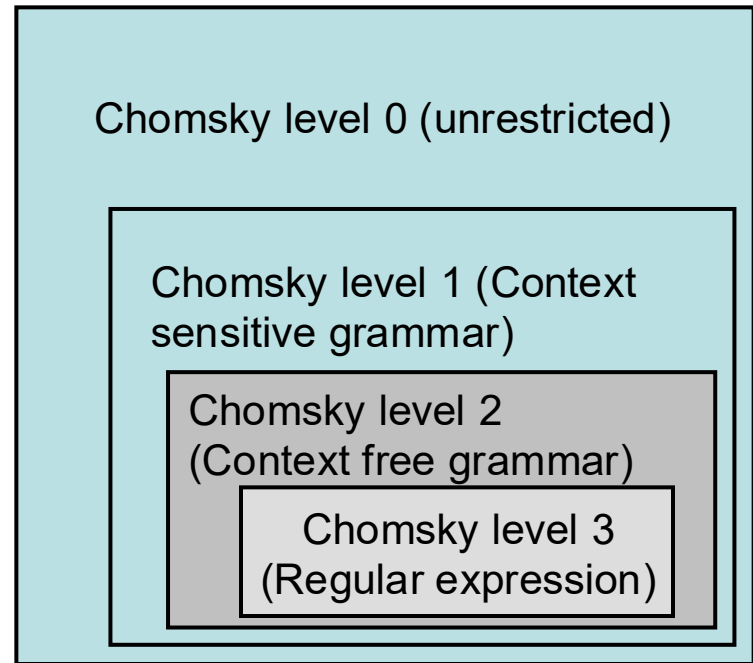
$R \rightarrow (a|b)(a|b|0|1)^*$

- Why two levels

- We think that way (sentence, word, character).
- besides, ... (see next slide)

Why use regular expression

- Every regular set is a context free language.
- Since we can use CFG to describe regular language, why using regular expression to define lexical syntax of a language? Why not using CFG to describe everything?
 - Lexical rules are simpler;
 - Regular expressions are concise and easier to understand;
 - More efficient lexical analyzers can be constructed from regular expression;
 - It is good to modularize compiler into two separate components.



BNF and EBNF

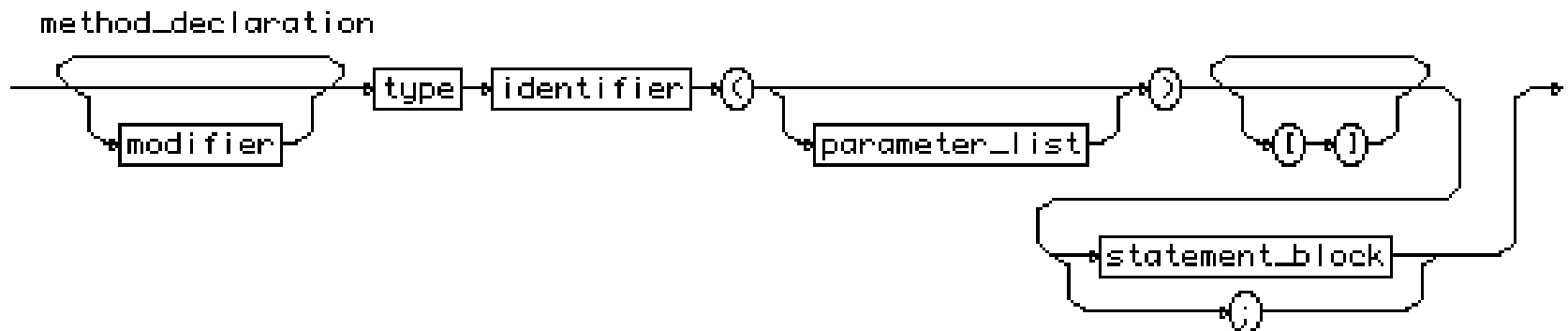
- BNF and EBNF are commonly accepted ways to express productions of a context-free grammar.
- BNF
 - Introduced by John Backus, first used to describe Algol 60. John won Turing award in 1977.
 - "Backus Normal Form", or "Backus Naur Form".
 - EBNF stands for Extended BNF.
- BNF format
 - lhs ::= rhs
 - Quote terminals or non-terminals
 - <> to delimit non-terminals,
 - bold or quotes for terminals, or 'as is'
 - Vertical bar for alternatives
 - There are many different notations, here is an example
 - `opt-stats ::= stats-list | EMPTY .`
 - `stats-list ::= statement | statement ';' stats-list .`

EBNF

- An extension of BNF, use regular- expression-like constructs on the right-hand-side of the rules:
 - write $[A]$ to say that A is optional
 - Write $\{A\}$ or A^* to denote 0 or more repetitions of A (ie. the Kleene closure of A).
- Using EBNF has the advantage of simplifying the grammar by reducing the need to use recursive rules.
- Both BNF and EBNF are equivalent to CFG
- Example:

BNF	EBNF
<pre>block ::= '{' opt-stats '}' opt-stats ::= stats-list EMPTY stats-list ::= statement statement ';' stats-list</pre>	<pre>block ::= '{' [stats-list] '}' stats-list ::= statement (';' statement)*</pre>

- **Java method_declaration**
- **method_declaration ::= { modifier } type identifier "(" [parameter_list] ")" { "[" "]" } (statement_block | ";")**



Languages, Grammars, and automata

Language Class	Grammar	Automaton
3	Regular	NFA or DFA
2	Context-Free	Push-Down Automaton
1	Context-Sensitive	Linear-Bounded Automaton
0	Unrestricted (or <i>Free</i>)	Turing Machine

→ Closer to machine

↓ More expressive

Another arithmetic expression example

(p1) $\text{exp} \rightarrow \text{exp} + \text{digit}$

(p2) $\text{exp} \rightarrow \text{exp} - \text{digit}$

(p3) $\text{exp} \rightarrow \text{digit}$

(p4) $\text{digit} \rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$

- the “|” means OR
- So the rules can be simplified as:
(P1-3) $\text{Exp} \rightarrow \text{exp} + \text{digit} | \text{exp} - \text{digit} | \text{digit}$
(p4) $\text{digit} \rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$

Derivation

- One step derivation ($\Rightarrow$ relation)

- If $A \rightarrow \gamma$ is a production, $\alpha A \beta$ is a string of terminals and variables, then

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

- Example:

$$9 - \text{digit} + \text{digit} \Rightarrow 9 - \text{digit} + 2$$

using rule P4 : $\text{digit} \rightarrow 2$

- Zero or more steps of derivation ($\Rightarrow^*$)

- Basis: $\alpha \Rightarrow^* \alpha$
- Induction: if $\alpha \Rightarrow^* \beta$, $\beta \Rightarrow \gamma$, then $\alpha \Rightarrow^* \gamma$.
- Example:

- $9 - \text{digit} + \text{digit} \Rightarrow^* 9 - \text{digit} + \text{digit}$
- $9 - \text{digit} + \text{digit} \Rightarrow^* 9 - 5 + 2$
- $9 - \text{digit} + \text{digit} \Rightarrow 9 - 5 + \text{digit} \Rightarrow 9 - 5 + 2$

- One or more steps of derivation ($\Rightarrow^+$)

- Example: $9 - \text{digit} + \text{digit} \Rightarrow^+ 9 - 5 + 2$

Example of derivation

- We can derive the string $9 - 5 + 2$ as follows:

$exp \Rightarrow \textcolor{red}{exp} + digit$	$(P1 : exp \rightarrow exp + digit)$
$\Rightarrow \textcolor{red}{exp} - digit + digit$	$(P2 : exp \rightarrow exp - digit)$
$\Rightarrow \textcolor{red}{digit} - digit + digit$	$(P3 : exp \rightarrow digit)$
$\Rightarrow 9 - \textcolor{red}{digit} + digit$	$(P4 : digit \rightarrow 9)$
$\Rightarrow 9 - 5 + \textcolor{red}{digit}$	$(P4 : digit \rightarrow 5)$
$\Rightarrow 9 - 5 + 2$	$(P4 : digit \rightarrow 2)$
$exp \Rightarrow^+ 9 - 5 + 2$	
$exp \Rightarrow^* 9 - 5 + 2$	

Left most and right most derivations

Left most derivation:

$$\begin{aligned} \text{exp} &\Rightarrow_{\text{lm}} \text{exp} + \text{digit} \\ &\Rightarrow_{\text{lm}} \text{exp} - \text{digit} + \text{digit} \\ &\Rightarrow_{\text{lm}} \text{digit} - \text{digit} + \text{digit} \\ &\Rightarrow_{\text{lm}} 9 - \text{digit} + \text{digit} \\ &\Rightarrow_{\text{lm}} 9 - 5 + \text{digit} \\ &\Rightarrow_{\text{lm}} 9 - 5 + 2 \\ \text{exp} &\Rightarrow_{\text{lm}}^+ 9 - 5 + 2 \\ \text{exp} &\Rightarrow_{\text{lm}}^* 9 - 5 + 2 \end{aligned}$$

Right most derivation:

$$\begin{aligned} \text{exp} &\Rightarrow_{\text{rm}} \text{exp} + \text{digit} \\ &\Rightarrow_{\text{rm}} \text{exp} + 2 \\ &\Rightarrow_{\text{rm}} \text{exp} - \text{digit} + 2 \\ &\Rightarrow_{\text{rm}} \text{exp} - 5 + 2 \\ &\Rightarrow_{\text{rm}} \text{digit} - 5 + 2 \\ &\Rightarrow_{\text{rm}} 9 - 5 + 2 \\ \text{exp} &\Rightarrow_{\text{rm}}^+ 9 - 5 + 2 \\ \text{exp} &\Rightarrow_{\text{rm}}^* 9 - 5 + 2 \end{aligned}$$

(p1) $\text{exp} \rightarrow \text{exp} + \text{digit}$

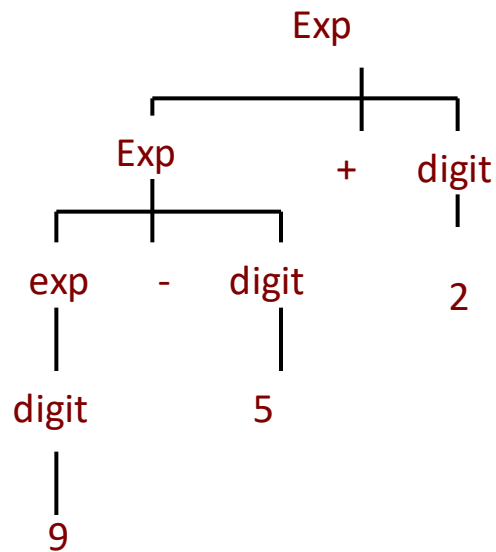
(p2) $\text{exp} \rightarrow \text{exp} - \text{digit}$

(p3) $\text{exp} \rightarrow \text{digit}$

(p4) $\text{digit} \rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$

Parse tree

- This derivation can also be represented via a *Parse Tree*.



9 - 5 + 2

(p1) $\text{exp} \rightarrow \text{exp} + \text{digit}$

(p2) $\text{exp} \rightarrow \text{exp} - \text{digit}$

(p3) $\text{exp} \rightarrow \text{digit}$

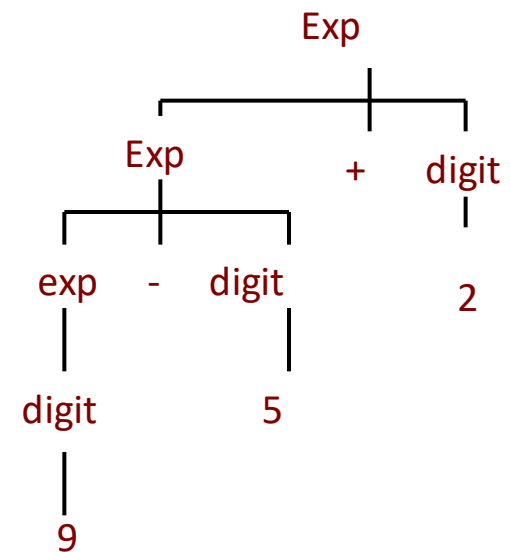
(p4) $\text{digit} \rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$

Tree terminologies

- Trees are collections of nodes, with a parent-child relationship.
 - A node has at most one parent, drawn above the node;
 - A node has zero or more children, drawn below the node.
- There is one node, the *root*, that has no parent. This node appears at the top of the tree
- Nodes with no children are called *leaves*.
- Nodes that are not leaves are *interior nodes*.

Formal definition of parse tree

- Parse tree shows the derivation of a string using a grammar.
- Properties of a parse tree:
 - The root is labeled by the start symbol;
 - Each leaf is labeled by a terminal or ϵ ;
 - Each interior node is labeled by a nonterminal;
 - If A is the nonterminal node and $X_1, \dots, X_n$ are the children nodes of A, then $A \rightarrow X_1 \dots X_n$ is a production.
- Yield of a parse tree
 - look at the leaves of a parse tree, from left to right, and concatenate them
 - Example: 9-5+2



The language of a grammar

- If G is a grammar, the language of the grammar, denoted as $L(G)$, is the set of terminal strings that have derivations from the start symbol.
- If a language L is the language of some context free grammar, then L is said to be a context free language.
- Example: the set of palindromes is a context free language.

Derivation and the parse tree

- The followings are equivalent:
 - $A \Rightarrow^* w$;
 - $A \Rightarrow_{lm}^* w$;
 - $A \Rightarrow_{rm}^* w$;
 - There is a parse tree with root A and yield w .

Applications of CFG

- Parser and parser generator;
- Markup languages.

Ambiguity of Grammar

- What is ambiguous grammar;
- How to remove ambiguity;



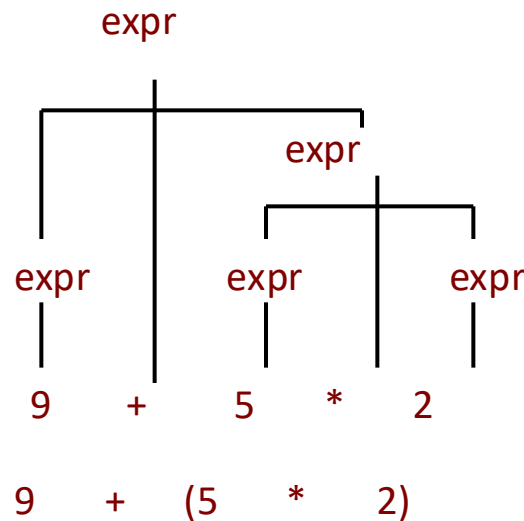
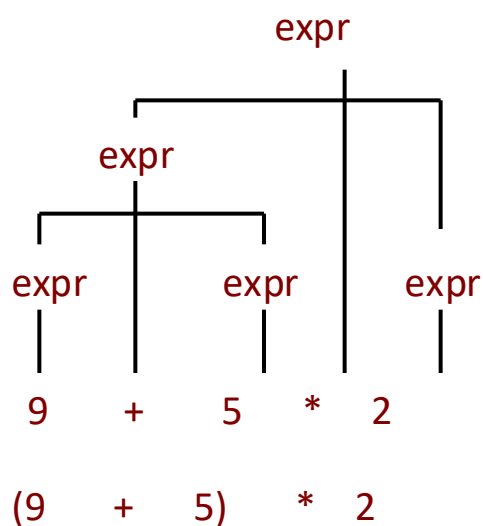
Example of ambiguous grammar

- Ambiguous sentence:
 - Fruit flies like a banana
- Consider a slightly modified grammar for expressions
$$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} * \text{expr} \mid \text{digit}$$
- Derivations of $9+5*2$
$$\begin{aligned} &\text{expr} \\ &\Rightarrow \text{expr} + \text{expr} \\ &\Rightarrow \text{expr} + \text{expr} * \text{expr} \\ &\Rightarrow {}_+ 9+5*2 \end{aligned}$$
$$\begin{aligned} &\text{expr} \\ &\Rightarrow \text{expr} * \text{expr} \\ &\Rightarrow \text{expr} + \text{expr} * \text{expr} \\ &\Rightarrow {}_+ 9+5*2 \end{aligned}$$
- There are different derivations

Ambiguity of a grammar

- Ambiguous grammar: produce more than one parse tree

$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{exp} * \text{expr} \mid \text{digit}$



- Problems of ambiguous grammar
 - one sentence has different interpretations

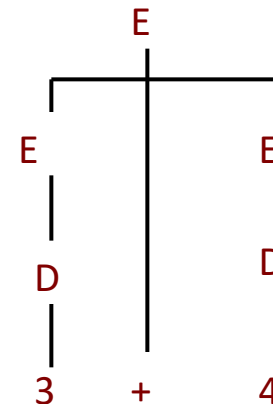
Several derivations can not decide whether the grammar is ambiguous

- Using the expr grammar, 3+4 has many derivations:

E
 $\Rightarrow$ E+E
 $\Rightarrow$ D+E
 $\Rightarrow$ 3+E
 $\Rightarrow$ 3+D
 $\Rightarrow$ 3+4

E
 $\Rightarrow$ E+E
 $\Rightarrow$ E+D
 $\Rightarrow$ E+4
 $\Rightarrow$ D+4
 $\Rightarrow$ 3+4

- Based on the existence of two derivations, we can not deduce that the grammar is ambiguous;
- It is not the multiplicity of derivations that causes ambiguity;
- It is the existence of more than one parse tree.
- In this example, the two derivations will produce the same tree



Derivation and parse tree

- In an unambiguous grammar, leftmost derivation will be unique; and rightmost derivation will be unique;
- How about ambiguous grammar?

E
 $\Rightarrow_{lm} E+E$
 $\Rightarrow_{lm} D+E$
 $\Rightarrow_{lm} 9+E$
 $\Rightarrow_{lm} 9+E^*E$
 $\Rightarrow_{lm} 9+D^*E$
 $\Rightarrow_{lm} 9+5^*E$
 $\Rightarrow_{lm} 9+5^*D$
 $\Rightarrow_{lm} 9+5^*2$

E
 $\Rightarrow_{lm} E^*E$
 $\Rightarrow_{lm} E+E^*E$
 $\Rightarrow_{lm} D+E^*E$
 $\Rightarrow_{lm} 9+E^*E$
 $\Rightarrow_{lm} 9+D^*E$
 $\Rightarrow_{lm} 9+5^*E$
 $\Rightarrow_{lm} 9+5^*D$
 $\Rightarrow_{lm} 9+5^*2$

- A string has two parser trees iff it has two distinct leftmost derivations (or two rightmost derivations).

Remove ambiguity

- Some theoretical results (bad news)
 - Is there an algorithm to remove the ambiguity in CFG?
 - the answer is no
 - Is there an algorithm to tell us whether a CFG is ambiguous?
 - The answer is also no.
 - There are CFLs that have nothing but ambiguous CFGs.
 - That kind of language is called ambiguous language;
 - If a language has one unambiguous grammar, then it is called unambiguous language.
- In practice, there are well-known techniques to remove ambiguity
- Two causes of the ambiguity in the expr grammar
 - the precedence of operator is not respected. “*” should be grouped before “+”;
 - a sequence of identical operator can be grouped either from left or from right. $3+4+5$ can be grouped either as $(3+4)+5$ or $3+(4+5)$.

Remove ambiguity

- Enforcing precedence by introducing several different variables, each represents those expressions that share a level of binding strength
 - factor: digit is a factor
 - term: factor * factor*factor is a term
 - expression: term+term+term ... is an expression
- So we have a new grammar:

$E \rightarrow T \mid E + T$

$T \rightarrow F \mid T * F$

$F \rightarrow D$

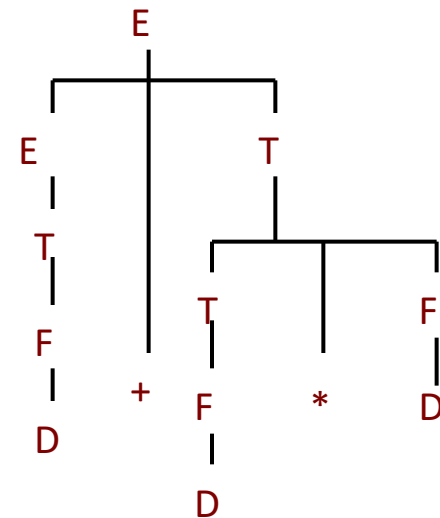
- Compare the original grammar:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow D$

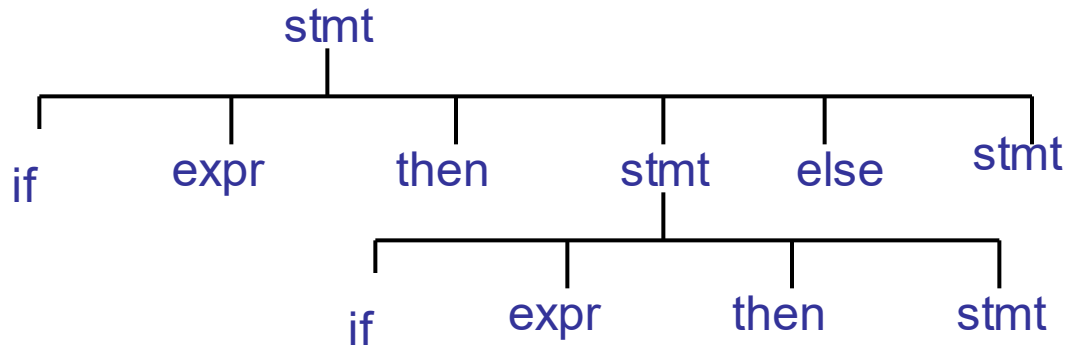
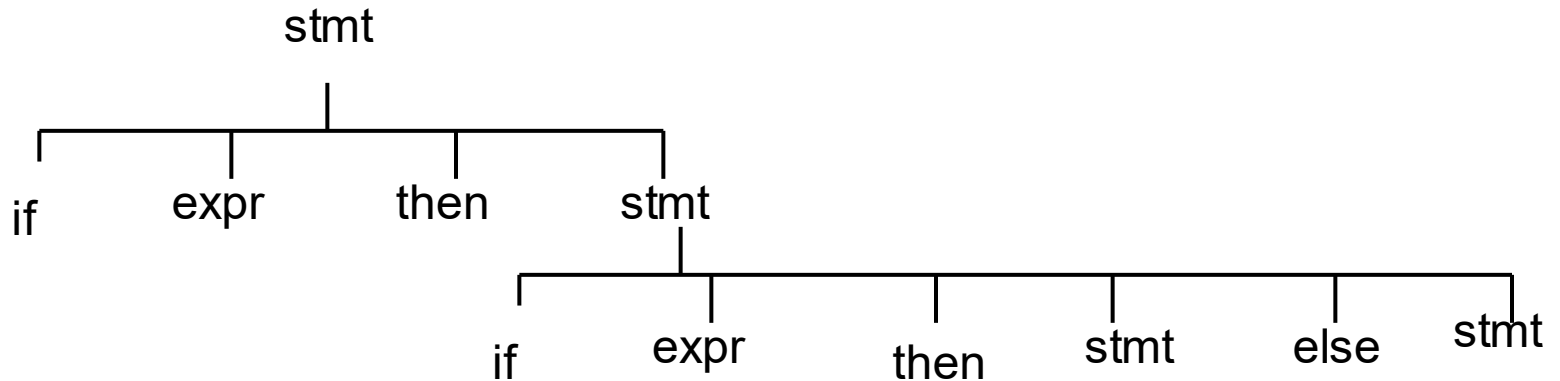
- The parser tree for $D + D * D$ is:



Another ambiguous grammar

Stmt $\rightarrow$ if expr then stmt
 | if expr then stmt else stmt
 | other

If E1 then if E2 then S1 else S2



Remove the ambiguity

- Match “else” with closest previous unmatched “then”
- How to incorporate this rule into the grammar?

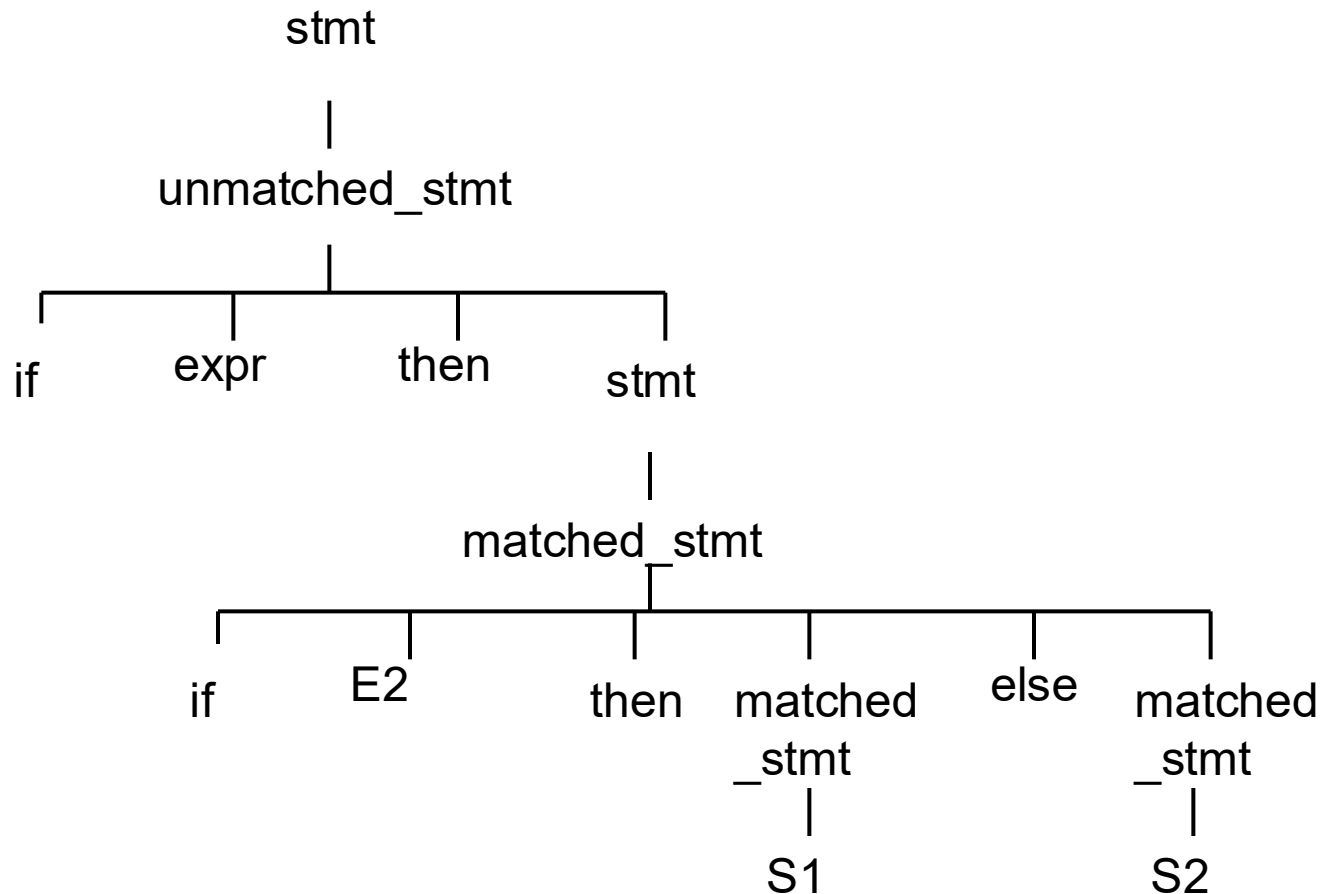
Stmt $\rightarrow$ if expr then stmt
 | if expr then **stmt** else stmt
 | other

stmt $\rightarrow$ matched_stmt | unmatched_stmt

matched_stmt $\rightarrow$ if expr then matched_stmt else matched_stmt
 | other

unmatched_stmt $\rightarrow$ if expr then stmt
 | if expr then matched_stmt else unmatched_stmt

The parse tree of if-stmt example



Readings in the dragon book

- 2.2.1. Grammar
- 2.2.2 Derivation
- 2.2.3 Parse tree
- 2.2.4 Ambiguity